Content

- Review of C programming
  - Macros,
  - Pointers, function pointers
  - and some common bugs
- C Memory Model
- System calls
  - Kernel mode, user mode
  - Interrupts, exceptions
- Error handling
- Debugging: GDB

Based on

https://github.com/angrave/SystemProgramming/wiki

Other resources:
Macro directives
- https://gcc.gnu.org/onlinedocs/cpp/Index-of-Directives.html#Index-of-Directives

struct packing:
- http://www.catb.org/esr/structure-packing/

GDB guide:
- https://darkdust.net/files/GDB%20Cheat%20Sheet.pdf

Linux terminal intro-guide:
- https://ubuntu.com/tutorials/command-line-for-beginners#1-overview

Intro to C

```
$ cat main.c
#include <stdio.h>
int main() {
     printf("Hello World!\n");
     return 0;
}
```

```
$ gcc main.c -o main
$ ./main
Hello World!
$
```

- `$ gcc -E main.c > main.i`
  - -E stops compiler after running preprocessor
- `$ gcc -S main.c`
  - -S stops compiler after assembly file generated
  - Assembler uses “.s” file to generate an object file
- `$ gcc -c main.c`
  - Generates object file main.o
  - main.o has undefined symbols like printf we don’t know where it is placed
  - `$ nm main.o`
- `$ gcc -o main main.c`
  - Generates executable main
  - printf does not have a value yet until the program is loaded
  - `$ nm main.o`

Preprocessor

```
/* Before preprocessing */
#define MAX_LENGTH 10
char buffer[MAX_LENGTH]

/* After preprocessing */
char buffer[10]
```

- Preprocessing is a copy and paste operation that the compiler performs before actually compiling the program.

- trying to redefine the internals of the C language with a preprocessor may be impossible.
- they can’t be infinitely - there is a bounded depth where they need to stop.

Macros- object like macros

An *object-like macro* is a simple identifier which will be replaced by a code fragment.

```
#define NUMBERS 1, \
                2, \
                3
int x[] = { NUMBERS };
/*after preprocessing
   → int x[] = { 1, 2, 3 };
*/

#define TABLESIZE BUFSIZE
#define BUFSIZE 1024
/*
TABLESIZE
     → BUFSIZE
     → 1024
*/
```

```
#define BUFSIZE 1020
#define TABLESIZE BUFSIZE
#undef BUFSIZE
#define BUFSIZE 37
```

https://gcc.gnu.org/onlinedocs/cpp/Macros.html#Macros

Macros- function like macros

A *function-like macros* **looks like** a function call, but a macro.

```
#define lang_init()  c_init()
lang_init()

/*after preprocessing
   → c_init()
*/

#define lang_init ()    c_init()
lang_init()

/*
→ () c_init()()
*/
```

https://gcc.gnu.org/onlinedocs/cpp/Macros.html#Macros

Macros with arguments

```
#define min(X, Y)  ((X) < (Y) ? (X) : (Y))

x = min(a, b);
/* →  x = ((a) < (b) ? (a) : (b));*/

y = min(1, 2);
/* →  y = ((1) < (2) ? (1) : (2)); */

z = min(a + 28, *p);
/* →  z = ((a + 28) < (*p) ? (a + 28) : (*p)); */
```

```
#define foo(x) x, "x"
foo(bar)

/*
→ bar, "x"
*/
```

https://gcc.gnu.org/onlinedocs/cpp/Macros.html#Macros

Macro pitfalls-misnesting

```
#define twice(x) (2*(x))
#define call_with_1(x) x(1)
call_with_1 (twice)

/*after preprocessing
    → twice(1)
     → (2*(1))
*/
```

```
//the use of unbalanced open parentheses in a
macro body is just confusing, and should be
avoided.
#define strange(file) fprintf (file, "%s %d",
…
strange(stderr) p, 35)

/*→ fprintf (stderr, "%s %d", p, 35)
*/
```

https://gcc.gnu.org/onlinedocs/cpp/Macros.html#Macros

Macro pitfalls-Operator Precedence Problems

```
#define ceil_div(x, y) (x + y - 1) / y


a = ceil_div (b & c, sizeof (int));


/*after preprocessing
    → a = (b & c + sizeof (int) - 1) / sizeof (int);
*/
```

Solution: each occurrence of a macro argument name should have parentheses around it.

```
/*What is the problem here?*/
#define min(a,b) a < b ? a : b

int main() {
     int x = 4;
     if(min(x++, 5)) printf("%d is six", x);

     int r = 10 + min(99, 100);
     printf("r is %d", r);
     return 0;
}
```

https://gcc.gnu.org/onlinedocs/cpp/Macros.html#Macros

Macro pitfalls-duplication side effects

```
/*What is the problem here?*/
#define min(X, Y)  ((X) < (Y) ? (X) : (Y))

int main() {
     int next = min (x + y, foo (z));
/*after preprocessing
    → next = ((x + y) < (foo (z)) ? (x + y) : (foo (z)));
the expression foo (z) has been substituted twice into the macro expansion.
*/
     return 0;
}
```

https://gcc.gnu.org/onlinedocs/cpp/Macros.html#Macros

Macro pitfalls-swallowing semicolon

```
#define SKIP_SPACES(p, limit)  \
{ char *lim = (limit);         \
  while (p < lim) {            \
    if (*p++ != ' ') {         \
      p--; break; }}}


SKIP_SPACES (p, lim);
/* “;” causes problems before else */

if (*p != 0)
  SKIP_SPACES (p, lim);
else …
```

```
/*solution*/
#define SKIP_SPACES(p, limit)     \
do { char *lim = (limit);         \
     while (p < lim) {            \
       if (*p++ != ' ') {         \
         p--; break; }}}          \
while (0)

SKIP_SPACES (p, lim);
/*after preprocessing
    expands into

do {…} while (0);
*/
```

https://gcc.gnu.org/onlinedocs/cpp/Macros.html#Macros

Macro conditionals

```
int main() {
     #ifdef __GNUC__
     return 1;
     #else
     return 0;
     #endif
}

$ gcc -E main.c >main.i
…
int main() {
 return 1;
}
```

In different compiler you would get return 0

C keywords

extern, break, const, continue

do{} while{};

enum

for (initialization; check; update) {}

if{} else{}

inline

goto (do not use in your programs)

restrict

return

signed, unsigned

sizeof -an operator

static

struct, union, typedef

switch case

C operators

In order of precedence

`[],`

`->,`

`.,`

`+/-a,`

`*a,`

`&a,`

`++, --,`

`sizeof,`

arithmetic operators `{+,-,*,%,/}`,

`>>/<<,`

`<=/>=,`

`==/!=, &&, ||,`

`!, &, |, ~,`

`?:,`

`a,b`

C Data types and Operators

Char, short (short int), int, long (long int), float, double
Fixed width integers
[**u**]**int***width***_t**

```
char a; /*1-byte data*/
short a; /*at least 2 bytes*/

int a; /*at least 2/4 bytes*/
long a;  /*at least 4/8 bytes*/
float a; /*IEEE-764 single precision floating point: 4 bytes*/
double a; /*IEEE-764 single precision floating point: 8 bytes*/
```

Shift operators

```
unsigned short uns = -127; // 1111111110000001
short sig = 1; // 0000000000000001
uns << 2; //1111111000000100
sig << 2; // 0000000000000100
uns >> 2; // 1111111111100000
sig >> 2; // 0000000000000000
```

In right shift;

- If the operand on the left is signed, then the integer is sign-extended. This means that if the number has the sign bit set, then any shift right will introduce 1s on the left.
- If the number does not have the sign bit set, any shift right will introduce zeros on the left.
- If the operand is unsigned, zeros will be introduced on the left either way.

Pointers

Declarations

```
int *ptr1;
char *ptr2;
int* ptr3, ptr4; /* ptr4 is a regular int */

int *ptr3, *ptr4; /*both pointers*/

struct person *ptr;
```

Reading/writing with pointers

```
*ptr = 0; /* Writes some memory.*/

int a = *ptr * 2 /*reads from memory*/
```

Pointer Arithmetic

```
char *ptr = "Hello"; /* ptr holds the memory location of ‘H’*/

ptr += 2; /* ptr now points to the first ‘l’*/
```

```
char *ptr = "ABCDEFGH";
int *bna = (int *) ptr;

bna +=1; /* Would cause iterate by one integer space (i.e 4 bytes on some systems) */

ptr = (char *) bna;
printf("%s", ptr);

/*
POSIX standards forbid arithmetic on void pointers!
*/
```

Void pointer

```
void *give_me_space = malloc(10);

char *string = give_me_space;
```

A pointer without a type.

Void pointers are used when either the datatype is unknown or when interfacing C code with other programming languages without APIs

POSIX standards forbid arithmetic on void pointers.

C strings

```
char* str = "constant"
```

- the string **`"constant"`** is stored in the data section.
- Depending on your architecture, it is read-only,
- any attempt to modify the string will cause a SEGFAULT.

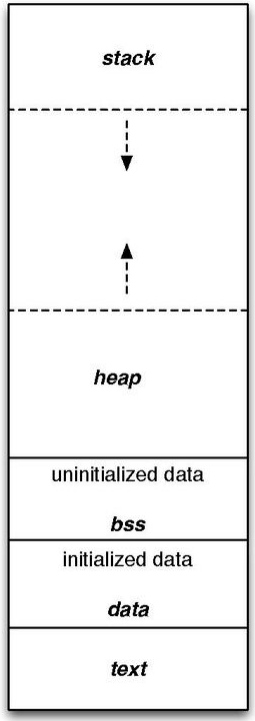
```
char str[] = "mutable"
```

- This is put in the global scope or the function scope for the data segment or the stack respectively

With malloc's space, you can change that string to be whatever they want.

Don't forget '\0' to terminate a string!!!

Program memory layout



stack:

- a dynamically growing and shrinking segment containing stack frames.
- One stack frame is allocated for each currently called function.
  - A frame stores the function's local variables (so-called automatic variables), arguments, and return value.

Heap:

- an area from which memory (for variables) can be dynamically allocated at run time.
- The top end of the heap is called the **program break.**

text:

- code segment, executable instructions

Data segment:

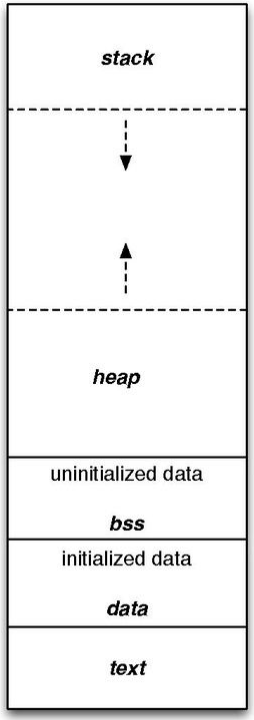
- initialized global and static variables

Bss segment:

- all global variables and static variables that are initialized to zero
- or do not have explicit initialization in source code.
- The whole segment is initialized to 0 at the start.

char arrays vs string constants

```
char array[] = "Hi!"; // array contains a mutable copy
strcpy(array, "OK");

char *ptr = "Can't change me"; // ptr points to some immutable memory
strcpy(ptr, "Will not work");


char *str1 = "Bhuvy likes books";
char *str2 = "Bhuvy likes books";
/*two string may reside in the same memory*/


/*however, char arrays must reside in different locations*/
char arr1[] = "Bhuvy also likes to write";
char arr2[] = "Bhuvy also likes to write";
```



String arrays vs string constants

```
/*Common ways to initialize a string*/
#include <stdio.h>

int main() {
        char *s0 = "abc";
        char *s1 = "abc";
        char s2[] = "abc";
        char s3[]= "abc";
        char s4[] = {'a', 'b', 'c', '\0'};
        printf("%p, %s\n", s0, s0);
        printf("%p, %s\n", s1, s1);
        printf("%p, %s\n", s2, s2);
        printf("%p, %s\n", s3, s3);
        printf("%p, %s\n", s4, s4);
}
```

```
/* address and contents
0x563610d0f004, abc
0x563610d0f004, abc
0x7ffc82bc7d9c, abc
0x7ffc82bc7da0, abc
0x7ffc82bc7da4, abc
```

Constant pointer vs Pointer to constant

```
/*a constant variable*/
const int a = 5;
const int b = 10;

/*a pointer to a constant integer*/
const int* ptr;

ptr = &a;
*ptr = 10; /*cannot do*/
```

```
/*a constant pointer to int*/
int c = 15, d = 20;
int *const ptr2 = &c;

/*cannot change the pointer value*/
ptr2 = &d;

/*constant pointer to a constant*/
const int *const ptr3 = &a;
```

const tells the compiler that the variable cannot be written (is constant).
This will result in a compile-time error: This has no influence on the memory space where the variable is located. In Run-time it may change its value.

Pointers to functions

```c
#include <stdio.h>
int intcmp(int *a, int *b){
   return *a > *b;
}
int main(){
   int x = 4, y = 5;
   /*declaration*/
   int (*comp)(void *, void *);

   /*assignment with type casting*/
   comp = (int (*)(void *, void *)) &intcmp;

   /*function call*/
   int z = (*comp)(&x, &y);

   printf("%d\n", z);
   return 0;
}
```

`int (*comp) (void *, void*)`

- `comp` is a pointer to a function
- The function should have two `void*` arguments
- The function returns `int`

`int *comp(void *, void*)/*WRONG*/`

- `comp` is a function
  - returning an int pointer

Pointers to functions are often used to implement **polymorphism** in “C”.

Structs and pointers

```c
#include <stdio.h>
typedef struct{
    int a1;
    int a2;
} pair;
```

```c
int main(){
    pair obj;
    pair zeros;
    zeros.a1 = 0;
    zeros.a2 = 0;

    pair *ptr = &obj;

    obj.a1 = 1;
    obj.a2 = 2;

    *ptr = zeros;
    printf("a1: %d, a2: %d\n", ptr->a1, ptr->a2);
    return 0;
}
```

Structs and pointers

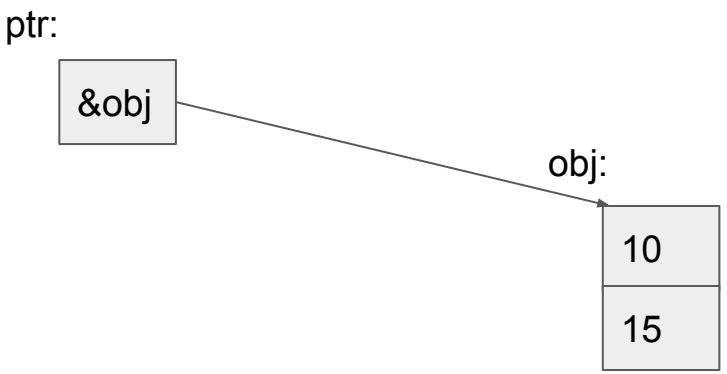
```c
#include <stdio.h>
typedef struct {
    int a1;
    int a2;
} pair;
```

```c
int main() {
    pair obj = {10, 15};
    pair zeros = {0, 0};
    obj = zeros;

    pair *ptr = &zeros;
    obj.a1 = 1;
    obj.a2 = 2;

    printf("a1: %d, a2: %d\n", ptr->a1, ptr->a2);
    return 0;
}
```

```c
#include <stdio.h>
typedef struct{
   int a1;
   int a2;
} pair;
int main(){
   pair obj = {10, 15};
   pair *ptr = &obj;

   /*what is the value of ptr2????*/
   pair *ptr2 = ptr + 1;

   /*ptr2 is the next struct, since there is no next, it is invalid*/
```

C memory model

```c
#include <stdio.h>
typedef struct{
    int a1;
    int a2;
} pair;
int main(){
    pair obj = {10, 15};
    int *ptr = (int*)&obj;
    printf("a1: %d, a2: %d\n", *ptr, *(ptr+1));
    return 0;
}
```



C memory model

```c
#include <stdio.h>
int main(){
   struct contact{
       char firstname[20];
       char lastname[20];
       unsigned int phone;
   };

   struct contact bhuvan;
   printf(" %p\n %p\n %p\n %p\n",
          &bhuvan,
          &bhuvan.firstname,
          &bhuvan.lastname,
          &bhuvan.phone);
}
```

Offset of a variable is the starting address of the variable's memory

In memory we expect

```
&bhuvan                 // 0x100
&bhuvan.firstname       // 0x100 = 0x100+0x00
&bhuvan.lastname        // 0x114 = 0x100+0x14
&bhuvan.phone           // 0x128 = 0x100+0x28
```

C memory model: Alignment requirements

The Lost Art of Structure Packing

C memory model: Alignment requirements

On modern processors,

- basic data types in memory is laid out in a way to make memory accesses faster.

- the basic C datatypes on a **vanilla ISA(instruction set architectures)** doesn't normally start at arbitrary byte addresses in memory.
  - These ISAs include Intel, ARM, and RISC-V

each type except char has an alignment requirement (**self-aligned)**;

- **chars**
  - can start on any byte address,
- **2-byte shorts**
  - must start on an even address,
- **4-byte ints** or **floats**
  - must start on an address divisible by 4,
- **8-byte longs** or **doubles**
  - must start on an address divisible by 8.
- **Pointers**
  - Based on word size
  - 4-bytes for 32-bit or 8 bytes for 64 bit
- **Signed** or **unsigned**
  - makes no difference.

C memory model: Padding

```c
char *p;
char c;
int x;
```

```c
/* the 4-byte alignment requirement of x
forces a gap in the layout;*/
char *p;      /* 4 or 8 bytes */
char c;       /* 1 byte */
char pad[3];  /* 3 bytes */
int x;        /* 4 bytes */
```

```c
char *p;
char c;
short x;
```

```c
/* the 4-byte alignment requirement of x
forces a gap in the layout;*/
char *p;      /* 4 or 8 bytes */
char c;       /* 1 byte */
char pad[1];  /* 1 bytes */
short x;        /* 2 bytes */
```

```c
char c;
char *p;
int x;
```

```c
/*the actual memory layout*/
char c;
char pad1[M];
char *p;
char pad2[N];
int x;
```

what can we say about M and N?

N is zero, M depends on the alignment of c.

- If the compiler happened to map c to the last byte of a machine word, the next byte (the first of p) would be the first byte of the next one and properly pointer-aligned. M would be zero.

Structure alignment and packing

```
struct foo1 {
    char *p;
    char c;
    long x;
};
```

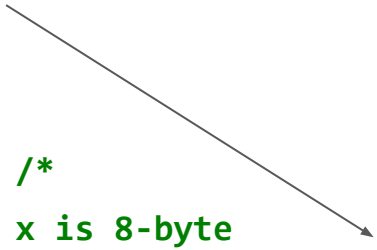
a struct instance will have the alignment of its widest scalar member.

```
/*
x is 8-byte
struct foo1 will have 8 byte alignment*/
struct foo1 {
    char *p;      /* 8 bytes */
    char c;       /* 1 byte */
    char pad[7]; /* 7 bytes */
    long x;       /* 8 bytes */
};
```

Structure alignment and packing

```c
struct foo2 {
    char c;
    char *p;
    long x;
};
```

Changing the order of the members will change the alignment



```c
/*
x is 8-byte
struct foo2 will have 8 byte alignment*/
struct foo2 {
    char c;       /* 1 byte */
    char pad[7]; /* 7 bytes */
    char *p;      /* 8 bytes */
    long x;       /* 8 bytes */
};
```

```c
#include <stdio.h>
struct foo3 {
   char *p; /* 8 bytes */
   char c;  /* 1 byte */
};

int main() {
   struct foo3 singleton;
   struct foo3 quad[4];

   /*on 64 bit machine, ???*/
   printf("%lu\n", sizeof(singleton));
   printf("%lu\n", sizeof(quad));
   return 0;
}
```

```c
sizeof(struct foo3)

struct foo3 {
   char *p;     /* 8 bytes */
   char c;      /* 1 byte */
   char pad[7];
};
```

More examples

```
struct foo4 {
   short s;     /* 2 bytes */
   char c;      /* 1 byte */
};
```

```
struct foo5 {
   char c;
   struct foo5_inner {
       char *p;
       short x;
   } inner;
};
```

sizeof(struct foo4)?

```
struct foo4 {
   short s;     /* 2 bytes */
   char c;      /* 1 byte */
   char pad[1];
};
```

sizeof(struct foo5)?

- char *p member in the inner struct forces the outer struct to be pointer-aligned as well as the inner.

```
struct foo5 {
   char c;           /* 1 byte*/
   char pad1[7];     /* 7 bytes */
   struct foo5_inner {
       char *p;      /* 8 bytes */
       short x;      /* 2 bytes */
       char pad2[6]; /* 6 bytes */
   } inner;
};
```

Structure reordering

```c
struct foo10 {
   char c;
   struct foo10 *p;
   short x;
};
```

```c
struct foo11 {
   struct foo11 *p;
   short x;
   char c;
};
```

```c
/*24 byte*/
struct foo10 {
   char c;            /* 1 byte */
   char pad1[7];      /* 7 bytes */
   struct foo10 *p;   /* 8 bytes */
   short x;           /* 2 bytes */
   char pad2[6];      /* 6 bytes */
};
```

```c
/*16 byte*/
struct foo11 {
   struct foo11 *p;   /* 8 bytes */
   short x;           /* 2 bytes */
   char c;            /* 1 byte */
   char pad[5];       /* 5 bytes */
};
```

- ➔ In general, ordering structure fields **by *increasing* size** minimize padding.
- ➔ C compilers don't do this automatically

Since c11, compile time static_assert

```c
#include <assert.h>
struct foo4 {
   short s;     /* 2 bytes */
   char c;      /* 1 byte */
};
static_assert(sizeof(struct foo4) == 4, "Check your assumptions");
```

There are compiler options to enforce this

```
struct picture{
      int height;
      pixel** data;
      int width;
      char* encoding;
}

/*with struct packing*/
struct picture{
      int height;
      char slop1[4];
      pixel** data;
      int width;
      char slop2[4];
      char* encoding;
}
```

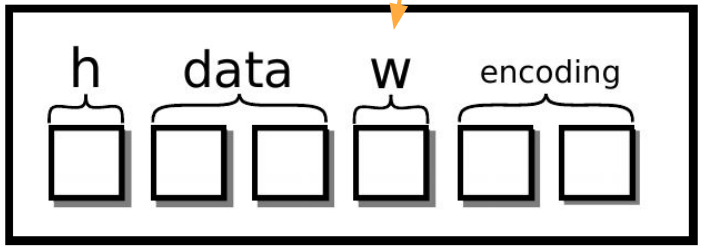


Figure 3.4: Six box struct

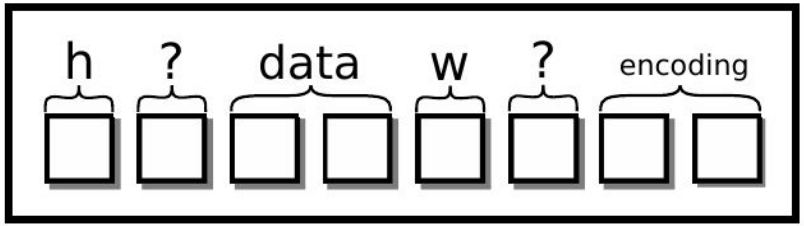


Figure 3.5: Eight box struct, two boxes of slop

Gcc attributes

```c
#include <stdio.h>
struct __attribute__((packed)) picture{
    char height;
    int** data;
    int width;
    char* encoding;
};

struct __attribute__((packed, aligned(4))) picture2{
    char height;
    int** data;
    int width;
    char* encoding;
};

int main(){
    printf("sizes: %ld-%ld\n", sizeof(struct picture), sizeof(struct picture2));
}
```

`packed` means it will use the smallest possible space for struct

`aligned(4)` its address can be divided by 4

sizes: 21-24

Flexible array member

```c
/*after c99 */
struct array {
        int size;
        int data[];
 };



/*or before c99, a “struct-hack”
with zero-length array*/
  struct array2 {
        int size;
        int data[0];
  };
```

Requires 1-malloc instead of 2-malloc

```c
#include <stdio.h>
#include <stdlib.h>
int main() {
    struct array {
        int size;
        char data[];
    };
    struct array *var =
        malloc(sizeof(*var) + 32 * sizeof(char));
    var->size = 32;
    var->data[0] = 5;
    var->data[31] = 2;

    printf("%lu-%lu\n", sizeof(struct array), sizeof(*var));
    printf("size:%d \n data[31]: %d\n",
        var->size, var->data[31]);
    free(var);
    return 0;
}
```

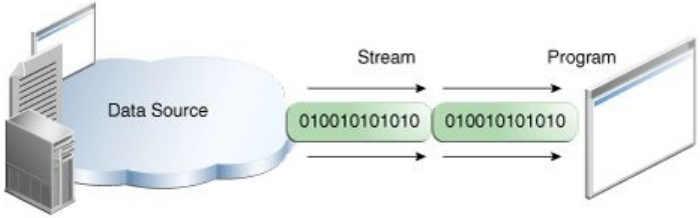
An Example

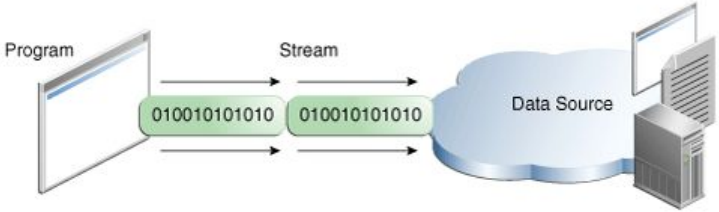
Put a frame around a given bitmap image

input

output

- Streams





Images are taken from doc.oracle.com

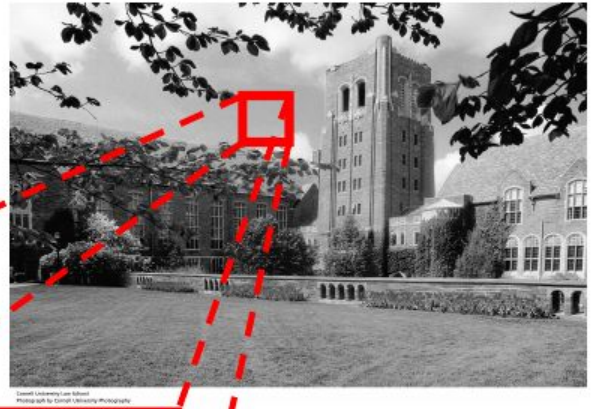
- *Default streams: `stdin, stdout, stderr`*
- ***`fopen(...)`***
- ***`fclose(...)`***
- **`fcloseall()`**
- To read
  - ***`fputc(...), putc(...), puthcar(...), fgetc(...), getc(...), getchar(...), getc(...),`***
  - ***Binary read: `fread()`***
- To write
  - ***`fputs(...),puts(...), fgets(...),`***

- *fscanf (...), fprintf (...)*

- **setvbuf (...)**
  - _IOFBF full buffer
  - _IOLBF line buffer
  - _IONBF without buffer
- *fflush(...)*

Grayscale image

1458-by-2084



150	149	152	153	152	155
151	150	153	154	153	156
153	151	155	156	155	158
154	153	156	157	156	159
156	154	158	159	158	161
157	156	159	160	159	162

8-bit
`unsigned char`
or
`uint8_t`

Color image

- RedGreenBlue
  - Red: a value in [0-255]
  - Green: a value in [0-255]
  - Blue: a value in [0-255]

Hex dump

- In Linux
  - $ **hexdump filename**

m.txt

```
badc
```

```
$ hexdump -c m.txt
0000000   b   a   d   c  \n
0000005
```

```
$ hexdump cute-animals32bit.bmp
0000000 4d42 68e2 0249 0000 0000 0046 0000 0038
0000010 0000 0f7d 0000 0973 0000 0001 0020 0003
0000020 0000 689c 0249 0b13 0000 0b13 0000 0000
0000030 0000 0000 0000 0000 ff00 0000 00ff ff00
0000040 0000 0000 0000 a500 a8b0 9b00 a0a8 9b00
0000050 9ea9 9100 96a1 9400 97a2 9d00 9ea8 9a00
0000060 97a2 9c00 97a2 a900 a4af a200 9faa 9d00
0000070 9ba5 a300 a2ac 9e00 9fa6 a500 a5ac a400
0000080 a3a8 a300 a1a7 9f00 a0a7 9e00 9da7 9e00
0000090 9ea5 9e00 9ca2 9b00 9ca1 9c00 9ea3 9f00
00000a0 a5ab a200 aab0 a000 abaf 9f00 a9ad a600
00000b0 acaf a000 a4a7 a500 a6a8 9b00 9c9e 9c00
00000c0 9fa1 9800 9b9d 8e00 9193 9200 9597 9200
00000d0 9398 9000 9196 9100 8f95 9100 8d93 9300
00000e0 8e92 9a00 9397 7c00 7677 8300 7d7e 8b00
***
```

Bitmap File Structure

Structure name	Optional	Size	Purpose	Comments
Bitmap file header	No	14 bytes	To store general information about the bitmap image file	Not needed after the file is loaded in memory
DIB header	No	Fixed-size (7 different versions exist)	To store detailed information about the bitmap image and define the pixel format	Immediately follows the Bitmap file header
Extra bit masks	Yes	3 or 4 DWORDs[5] (12 or 16 bytes)	To define the pixel format	Present only in case the DIB header is the BITMAPINFOHEADER and the Compression Method member is set to either BI_BITFIELDS or BI_ALPHABITFIELDS
Color table	Semi-optional	Variable-size	To define colors used by the bitmap image data (Pixel array)	Mandatory for color depths ≤ 8 bits
Gap1	Yes	Variable-size	Structure alignment	An artifact of the File offset to Pixel array in the Bitmap file header
Pixel array	No	Variable-size	To define the actual values of the pixels	The pixel format is defined by the DIB header or Extra bit masks. Each row in the Pixel array is padded to a multiple of 4 bytes in size
Gap2	Yes	Variable-size	Structure alignment	An artifact of the ICC profile data offset field in the DIB header
ICC color profile	Yes	Variable-size	To define the color profile for color management	Can also contain a path to an external file containing the color profile. When loaded in memory as "non-packed DIB", it is located between the color table and Gap1.

BMP file header

Offset hex	Offset dec	Size	Purpose
00	**0**	**2 bytes**	The header field used to identify the BMP and DIB file is 0x42 0x4D in hexadecimal, same as BM in ASCII. The following entries are possible: BM Windows 3.1x, 95, NT, ... etc. BA OS/2 struct bitmap array CI OS/2 struct color icon CP OS/2 const color pointer IC OS/2 struct icon PT OS/2 pointer
02	**2**	**4 bytes**	The size of the BMP file in bytes
06	**6**	**2 bytes**	Reserved; actual value depends on the application that creates the image
08	**8**	**2 bytes**	Reserved; actual value depends on the application that creates the image
0A	**10**	**4 bytes**	The offset, i.e. starting address, of the byte where the bitmap image data (pixel array) can be found.

DIP header

Offset (hex)	Offset (dec)	Size (bytes)	Windows BITMAPINFOHEADER[1]
0E	14	4	the size of this header (40 bytes)
12	18	4	the bitmap width in pixels (signed integer)
16	22	4	the bitmap height in pixels (signed integer)
1A	26	2	the number of color planes (must be 1)
1C	28	2	the number of bits per pixel, which is the color depth of the image. Typical values are 1, 4, 8, 16, 24 and 32.
1E	30	4	the compression method being used. See the next table for a list of possible values
22	34	4	the image size. This is the size of the raw bitmap data; a dummy 0 can be given for BI_RGB bitmaps.
26	38	4	the horizontal resolution of the image. (pixel per metre, signed integer)
2A	42	4	the vertical resolution of the image. (pixel per metre, signed integer)
2E	46	4	the number of colors in the color palette, or 0 to default to 2^n
32	50	4	the number of important colors used, or 0 when every color is important; generally ignored

```c
char fname[] = "cute-animals32bit.bmp";
FILE *f = fopen(fname, "rb");

if (f == NULL){
    perror("the file \" %s \"  is not found!", fname);
    return 0;
}

uint32_t offset, hsize, width, height;
uint16_t nplanes, nbits;
fseek(f, 10, SEEK_CUR);
fread(&offset, 4, 1, f);
fread(&hsize, 4, 1, f);
fread(&width, 4, 1, f);
fread(&height, 4, 1, f);
fread(&nplanes, 2, 1, f);
fread(&nbits, 2, 1, f);
printf("offset: %d, hsize: %d, \n"
        "width: %d, height: %d, nplanes: %d, bits: %d",
        offset, hsize, width, height, nplanes,
        nbits);
fclose(f);
```

Copy content[0-offset] to another file

```
/*storage for the data upto offset of image data*/
uint8_t *fheader = (uint8_t *)malloc(sizeof(uint8_t)*offset);

/*rewind to the beginning*/
rewind(infile);

/*read and write the header upto offset*/
fread(fheader, 1, offset, infile);
fwrite(fheader, 1, offset, outfile);
```

User defined types

- Struct
- Bitfield
- Enumeration
- typedef

Put frame to new file

```
uint32_t apixel = 0; /*storage for a pixel data*/
uint32_t FRAME_COLOR = 255<<1 | 255<<16 |255<<8; /*a-r-g-b-0*/

rewind(infile);
fseek(infile,offset,SEEK_CUR);/*go to image data*/

for(int i = height; i > 0; i--){
    for(int j = 0; j < width; j++){
        /*read a pixel (4byte)*/
         fread(&apixel, 4, 1, infile);

         /*write the pixel to outfile */
         if(j < 500 || i < 500 || i > height-500 || j> width-500){
             fwrite(&FRAME_COLOR, 4, 1, outfile);
         }else{
             fwrite(&apixel,4,1, outfile);
         }
    }
}
```

BGRX, the storage in bmp file https://en.wikipedia.org/wiki/BMP_file_format#Pixel_storage

Struct for 32-bit pixel data

- Image data is little endian!
  - Rgba in reverse order

```
struct pixel{
    unsigned char b;
    unsigned char g;
    unsigned char r;
    unsigned char a;
} apixel;
```

```
struct pixel{
    uint8_t b;
    uint8_t g;
    uint8_t r;
    uint8_t a;
} apixel;
```

Bit fields

```
struct pixel{
    unsigned int b:8;
    unsigned int g:8;
    unsigned int r:8;
    unsigned int a:8;
} apixel;
```

- You cannot get the address of bitfields!
  - ~~`&apixel.b`~~
- Their order may change
- Be careful whether the machine is little or big endian!

Another example

```
struct page{
   unsigned int pt_pres: 1;  /* page is present?          */
   unsigned int pt_write : 1;/* page is writable?         */
   unsigned int pt_user: 1;  /* is use level protection?     */
   unsigned int pt_pwt: 1;   /* write through for this page? */
   unsigned int pt_pcd: 1;   /* cache disable for this page? */
   unsigned int pt_acc: 1;   /* page was accessed?        */
   unsigned int pt_dirty : 1;/* page was written?            */
   unsigned int pt_mbz: 1;   /* must be zero                 */
   unsigned int pt_global: 1;/* should be zero in 586        */
   unsigned int pt_avail : 3;/* for programmer's use         */
   unsigned int pt_base: 20; /* location of page?            */
} pt_t;
```

taken from XINU OS

Example use

```
union page {
    uint32_t all;
    struct {
        unsigned int pt_pres : 1;   /* page is present?        */
        unsigned int pt_write : 1;  /* page is writable?           */
        unsigned int pt_user : 1;   /* is use level protection?    */
        unsigned int pt_pwt : 1;    /* write through for this page? */
        unsigned int pt_pcd : 1;    /* cache disable for this page? */
        unsigned int pt_acc : 1;    /* page was accessed?          */
        unsigned int pt_dirty : 1;  /* page was written?            */
        unsigned int pt_mbz : 1;    /* must be zero                 */
        unsigned int pt_global : 1; /* should be zero in 586        */
        unsigned int pt_avail : 3;  /* for programmer's use         */
        unsigned int pt_base : 20;  /* location of page?              */
    } __attribute__((packed)) page;
} __attribute__((packed)) pt_t;
pt_t.all = 5; // try 5 << 9;
printf("%u-%u-%u-%u \n", pt_t.page.pt_pres, pt_t.page.pt_write,
       pt_t.page.pt_user, pt_t.page.pt_avail);
```

Most machines are little endian.

```c
typedef union {
   uint32_t pixel;
   struct {
       uint8_t b;
       uint8_t g;
       uint8_t r;
       uint8_t a;      /* a or x in argb, xrgb   */
   }__attribute__((packed));
} Pixel;
```

```c
/*similar to this*/
char pixel[] = {0,0,0,0}; //{b,g,r,a}
```

```c
Pixel p;
p.pixel = 0x12345678;
/* integers in little endian: p.b = 78, p.g = 56, p.r = 34, p.a = 12 */
```

```
uint32_t FRAMECOLOR = 0 << 24 | 255 << 16 | 0 << 8 | 0; /*a-r-g-b*/
#define FRAMESIZE 50
Pixel p; /*storage for a pixel data*/
p.pixel = 0;
for (int i = height - 1; i >= 0; i--) {
    for (int j = 0; j < width; j++) {
        fread(&p.pixel, 4, 1, infile);

        /*write the pixel to outfile */
        if (j < FRAMESIZE || i < FRAMESIZE || i > height - FRAMESIZE ||
            j > width - FRAMESIZE) {
            fwrite(&FRAMECOLOR, 4, 1, outfile);
        } else {
            fwrite(&p.b, 1, 1, outfile);
            fwrite(&p.g, 1, 1, outfile);
            fwrite(&p.r, 1, 1, outfile);
            fwrite(&p.a, 1, 1, outfile);
        }
    }
}
```

C functions and System calls

Common C functions

POSIX mantra; “**Everything is a file**”

- Everything is a **file descriptor**
  - an integer.

```
int file_fd = open(...);

int network_fd = socket(...);

int kernel_fd = epoll_create1(...);
```

- These are all references to **the kernel’s file descriptor table (a table of objects)**.
- File descriptors are simply pointers to these objects.

Operations on these objects

- open, close, allocation, deallocation
  - are done through **system calls.**

The program interacts with these objects using the API specified through system calls and library functions.

System calls

User processes

- cannot perform privileged operations

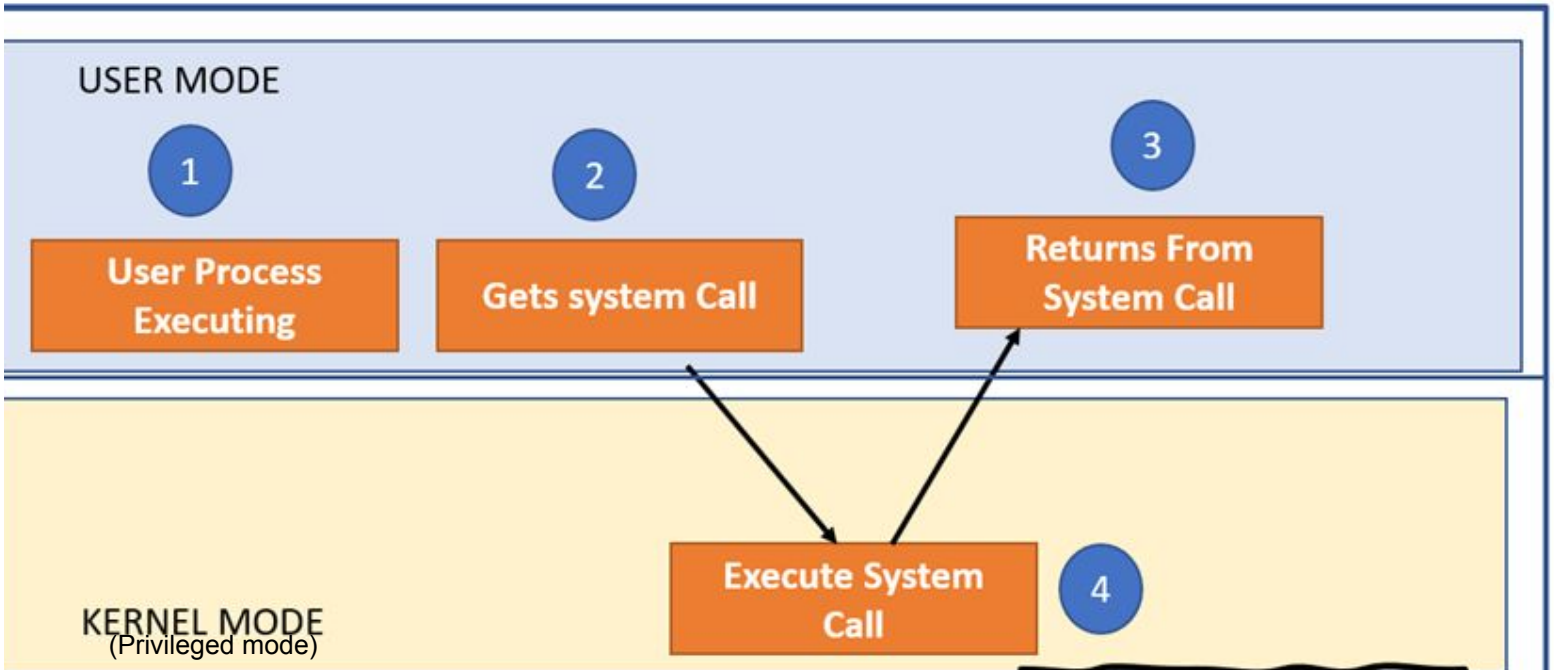
- request OS to do so on their behalf by issuing **system calls**

A **system call** is a request made by **an *active process (through traps or software interrupts)*** for a service performed by the kernel.

Examples:

- input/output (i.e., any movement of information to or from the combination of the CPU and main memory)
- creation of a new process.
- And many more https://man7.org/linux/man-pages/man2/syscalls.2.html

Why do we need a system call?



https://www.guru99.com/system-call-operating-system.html#2

How do we switch(transition) to kernel mode?

Arch/ABI	Instruction	System call #	Ret val	Ret val2	Error	Notes
alpha	callsys	v0	v0	a4	a3	1, 6
arc	trap0	r8	r0	-	-	
arm/OABI	swi NR	-	r0	-	-	2
arm/EABI	swi 0x0	r7	r0	r1	-	
arm64	svc #0	w8	x0	x1	-	
blackfin	excpt 0x0	P0	R0	-	-	
i386	**int $0x80**	**eax**	**eax**	**edx**	**-**	
ia64	break 0x100000	r15	r8	r9	r10	1, 6
m68k	trap #0	d0	d0	-	-	
microblaze	brki r14,8	r12	r3	-	-	
mips	syscall	v0	v0	v1	a3	1, 6
nios2	trap	r2	r2	-	r7	
…						
x86-64	**syscall**	**rax**	**rax**	**rdx**	**-**	**5**
x32	syscall	rax	rax	rdx	-	5
xtensa	syscall	a2	a2	-	-	

https://man7.org/linux/man-pages/man2/syscall.2.html

```c
#include <unistd.h>
int main(int argc, char *argv[]){
   write(1, "Hello World\n", 12); /* write "Hello World" to stdout */
   _exit(0);                      /* exit with error code 0 (no error) */
}
```

x86-32 equivalent assembly code

```
_start:
   movl $4, %eax   ; use the `write` [interrupt-flavor] system call
   movl $1, %ebx   ; write to stdout
   movl $msg, %ecx ; use string "Hello World"
   movl $12, %edx  ; write 12 characters
   int $0x80       ; make system call

   movl $1, %eax   ; use the `_exit` [interrupt-flavor] system call
   movl $0, %ebx   ; error code 0
   int $0x80       ; make system call
```

https://en.wikibooks.org/wiki/X86_Assembly/Interfacing_with_Linux

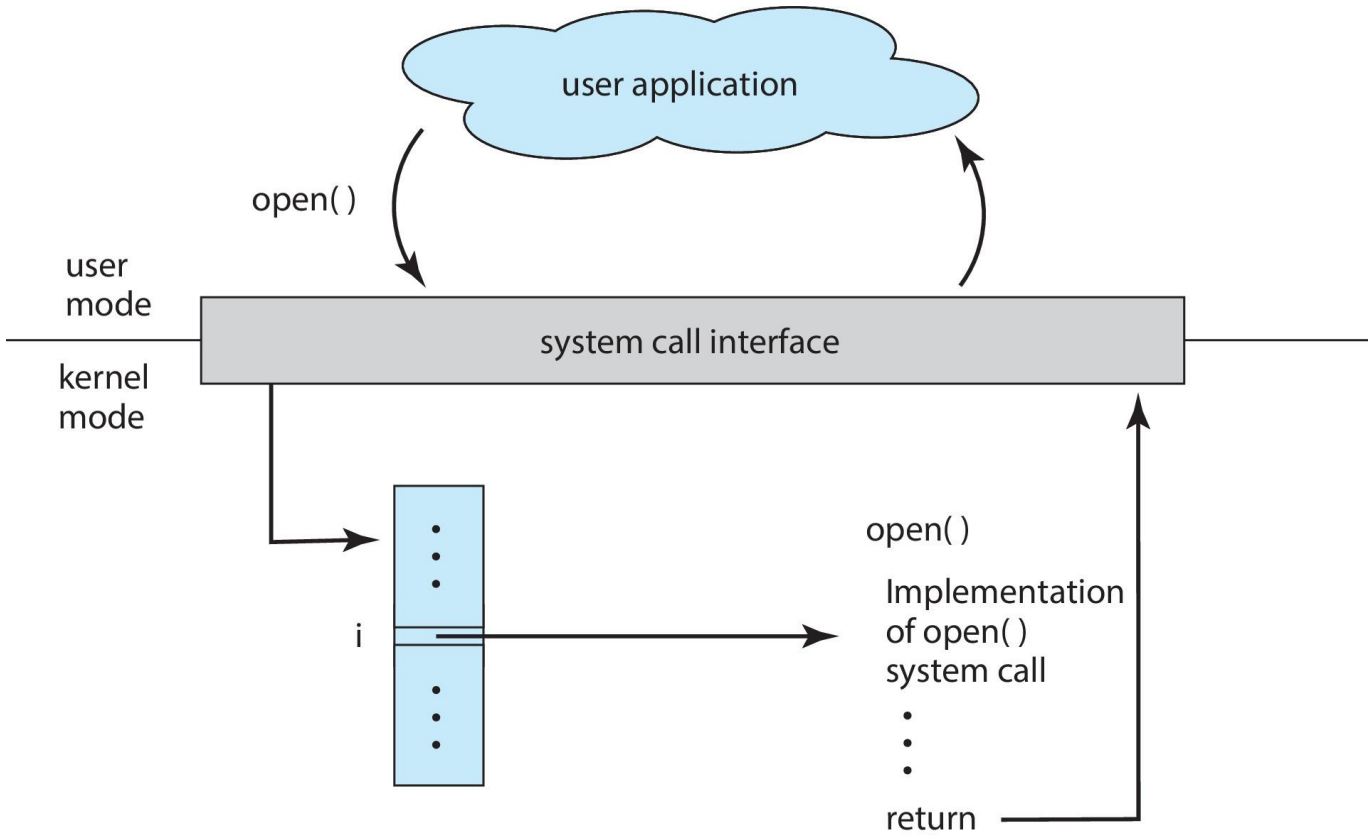
```c
#include <unistd.h>
int main(int argc, char *argv[]){
   write(1, "Hello World\n", 12); /* write "Hello World" to stdout */
   _exit(0);                      /* exit with error code 0 (no error) */
}
```

x86-64 equivalent assembly code

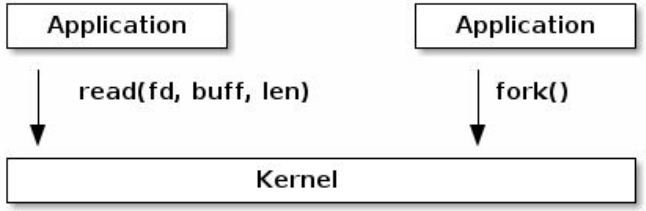
```
_start:
   movq $1, %rax    ; use the `write` [fast] syscall
   movq $1, %rdi    ; write to stdout
   movq $msg, %rsi  ; use string "Hello World"
   movq $12, %rdx   ; write 12 characters
   syscall          ; make syscall

   movq $60, %rax   ; use the `_exit` [fast] syscall
   movq $0, %rdi    ; error code 0
   syscall          ; make syscall
```

https://en.wikibooks.org/wiki/X86_Assembly/Interfacing_with_Linux

API – System Call – OS Relationship

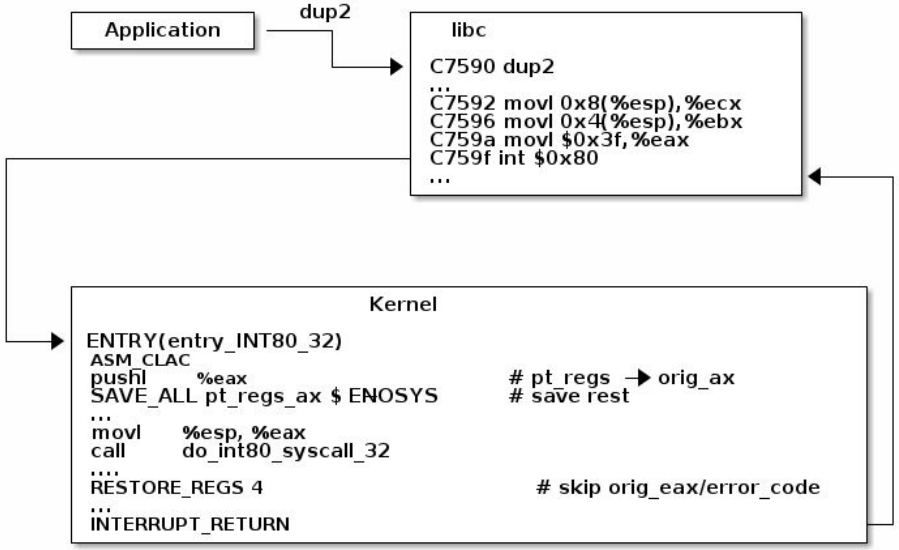


Example dup2() on x86



System calls are specific assembly instructions that

- setup information to identify the system call and its parameters
- trigger a kernel mode switch
- retrieve the result of the system call



https://linux-kernel-labs.github.io/refs/heads/master/lectures/syscalls.html

Some definitions

Interrupt

A signal to the kernel from a hardware or a software that indicates an event need “immediate” attention

- **Hardware interrupts:** the pressing of a key on the keyboard, a movement of the mouse.

- **Software interrupts (e.g. int 0x80)**
  - requests by an application program to operating systems
  - each interrupt is associated with an interrupt handler

★ CPU switch control to an interrupt handler
★ The execution of the program resumes after the interrupt is handled

trap vs exception vs interrupt

An exception is an unexpected event occurs because of **the execution of an instruction** (synchronous event).

- Division by zero
- **CPU detects this:**
- **Exception Types**
    - Traps,
        - Breakpoint, Trap, Syscalls, division by zero
    - faults,
        - Memory faults(Page fault),
    - aborts

Hardware interrupt is an unexpected event from outside the processor.

Software interrupt (e.g., int 0x80) is generated by the application programs(similar to traps).

https://stackoverflow.com/questions/3149175/what-is-the-difference-between-trap-and-interrupt

Summary and warnings

```
int file_fd = open(...);
```

- We got access to a file descriptor object.

```
write(file_fd, "Hello!", 6);
```

- writes some bytes to the file descriptor object that represents a file

A system call is an operation that the kernel carries out.

- First, the operating system prepares a system call.
- Next, the kernel executes the system call to the best of its ability in kernel space (a privileged operation).

★ System calls are expensive.
  - Their cost in terms of time and CPU cycles has recently been decreased, but try to use them as sparingly as possible.

★ this operation could fail for several reasons:
  - the file is no longer valid,
  - the hard drive failed,
  - the system was interrupted etc.

C system call-abstractions

Many C functions are abstractions that call the correct underlying system call

The implementations are different based on the platforms.

Common C functions

man pages are organized into sections:

Section 2 are system calls

```
man -S2 open
```

Section 3 are C libraries

```
man -S3 printf
```

Handling Errors

In Java and C++ errors handled through exceptions

Arguments against exceptions

- Exceptions make control flow harder to understand.
- Exception oriented languages need to keep stack traces and maintain jump tables.
- Exceptions may be complex objects.

There are a few arguments for exceptions as well

+ Exceptions can come from several layers deep.
+ Exceptions help reduce global state.
+ Exceptions differentiate business logic and normal flow.

C stderr, error messages

```c
#include <stdio.h>
#include <stdlib.h>
main(){
   int dividend = 20, divisor = 0, quotient;
   if (divisor == 0){
       fprintf(stderr, "Division by zero! Exiting...\n");
       exit(-1); /*=exit(EXIT_FAILURE)*/
   }

   quotient = dividend / divisor;
   fprintf(stderr, "Value of quotient : %d\n", quotient);

   exit(0); /*=exit(EXIT_SUCCES)*/
}
```

Handling Errors: errno

- C does not provide direct support for error handling (also known as exception handling).
- -1 and NULL are used to indicate errors
- In Linux, global variable errno is assigned to the code
- errno can be used to identify the problem

strerror() and **perror()** can be used to display the error message associated with **errno**

errno value	Error
1	/* Operation not permitted */
2	/* No such file or directory */
3	/* No such process */
4	/* Interrupted system call */
5	/* I/O error */
6	/* No such device or address */
7	/* Argument list too long */
8	/* Exec format error */
9	/* Bad file number */
10	/* No child processes */
11	/* Try again */
12	/* Out of memory */
13	/* Permission denied */

```c
#include <errno.h>
#include <stdio.h>
#include <string.h>
int main(){
   FILE *f = fopen("/does/not/exist.txt", "r");
   if (NULL == f){
       fprintf(stderr, "Errno is %d\n", errno);

       fprintf(stderr, "Description is %s\n", strerror(errno));

       perror("Description is ");
   }
}
```

Common Bugs in C Programs

```c
void mystrcpy(char *dest, char *src){
   // void means no return value
   while (*src)   {
       dest = src;  src++;  dest++;
   }
}
```

```c
while( *src ) {*dest = *src; src ++; dest++; }
*dest = *src;
```

```c
while( (*dest++ = *src++ )) {};/*bad coding practice!*/
/****************************************/
```

```c
int *p = malloc(sizeof(int));
free(p);
*p = 123; /* Oops! - Dangling pointer!
Writing to memory we don't own anymore */

free(p);  /* Oops! - Double free! */
```

```c
p = NULL; // No dangling pointers
```

```c
/*returning automatic variables*/
int *f(){
   int result = 42;
   static int imok;
   return &imok;   /* OK - static variables are not on the stack */
   return &result; /* Not OK*/
}
```

```
/*in sufficient memory*/
struct User {
   char name[100];
};
typedef struct User user_t;
user_t *user = (user_t *)malloc(sizeof(user));
```

In the above example, we needed to allocate enough bytes for the struct. Instead, we allocated enough bytes to hold a pointer.

```
/*the correct code*/

user_t *user = (user_t *)malloc(sizeof(user_t));
```

```c
/*buffer overflow/underflow*/
#define N (10)
int i = N, array[N];
for( ; i >= 0; i--) array[i] = i;



gets(array); /*WRONG!: Let’s hope the input is shorter than my array!*/

/*Do NOT use gets, very unsafe!!!*/
/*alternative fgets*/
```

```c
/*Strings require strlen(s)+1 bytes*/
/* returns a copy of ’input’ */
char *strdup(const char *input){
   char *copy;
   copy = malloc(sizeof(char *)); /* nope! this allocates space for
    a pointer, not a string */

   copy = malloc(strlen(input)); /* Almost...but what about the
   null terminator? */

   copy = malloc(strlen(input) + 1); /* That’s right. */
   strcpy(copy, input);              /* strcpy will provide the null terminator */
   return copy;
}
```

```c
/*Using uninitialized variables*/
int myfunction(){
   int x;
   int y = x + 2;

/*Assuming Uninitialized memory will be zeroed:
Automatic (temporary variables) and heap allocations may contain
random bytes or garbage.*/
void myfunct(){
   char array[10];
   char *p = malloc(10);
```

```c
/*Equality vs equal*/
int p1, p2;
p1 = p2 = 0;

int answer = 3; // Will print out the answer.
if (answer = 42){
   printf("The answer is %d", answer);
}

if (42 = answer){ /*compiler  will catch the
error*/
   printf("The answer is %d", answer);
}
```

```c
/*Extra Semicolons*/
for (int i = 0; i < 5; i++);
   printf("Printed once");

while (x < 10);
x++; // X is never incremented
```

Using a debugger

GDB is short for the GNU Debugger.

A program that helps you track down errors by interactively debugging

them.

It can start and stop your program, look around, and put in ad hoc constraints and checks.

-g option add debugging information to your program

```
gcc –g –o hello hello.c
```

Intro to gdb

```
gdb programname
```

or

```
gdb programname core
```

to examine the coredump as if the program was running and had just faulted.

In GDB

run or **r: runs the program**

bt: backtrace when a segfault occurs

p: print the value of a variable

Google gdb cheat sheet to get a list of commands in gdb

Setting breakpoints programmatically

```
int main() {
      int val = 1;
      val = 42;
      asm("int $3"); /* generates interrupt*/
      val = 7;
}
```

```
$ gcc main.c -g -o main
$ gdb --args ./main
(gdb) r
[...]
Program received signal SIGTRAP,
Trace/breakpoint trap.
main () at main.c:6
6
val = 7;
(gdb) p val
$1 = 42
```

gdb progname
(gdb)
Then set a breakpoint in the main function.
(gdb) break main
A breakpoint is a marker in your program that will make the program stop and return control back to gdb.
Now run your program.
(gdb) run
If your program has arguments, you can pass them after run.

Your program will stop when it reaches “main()”
gdb>

(gdb) step
It will run the next line of code and stop. If it is a function call, it will enter into it
(gdb) next
It will run the next line of code and stop. If it is a function call, it will not enter the function and it will go through it.

(gdb) break functionname

Set a breakpoint in a function

(gdb) break 66

It will set a break point in line 66 of the current file.

(gdb) break filename:linenumber

It will set a break point at a line in a specific file.

(gdb) print var

Prints the value of a variable.

E.g.

```
(gdb) print i
$1 = 5
(gdb) print s1
$1 = 0x10740 "Hello"
(gdb) print stack[2]
$1 = 56
(gdb) print stack
$2 = {0, 0, 56, 0, 0, 0, 0, 0, 0, 0}
(gdb)
```

Memory leaks (demo with valgrind)

valgrind package (eg, in debian sudo apt install valgrind)

https://valgrind.org/docs/manual/quick-start.html